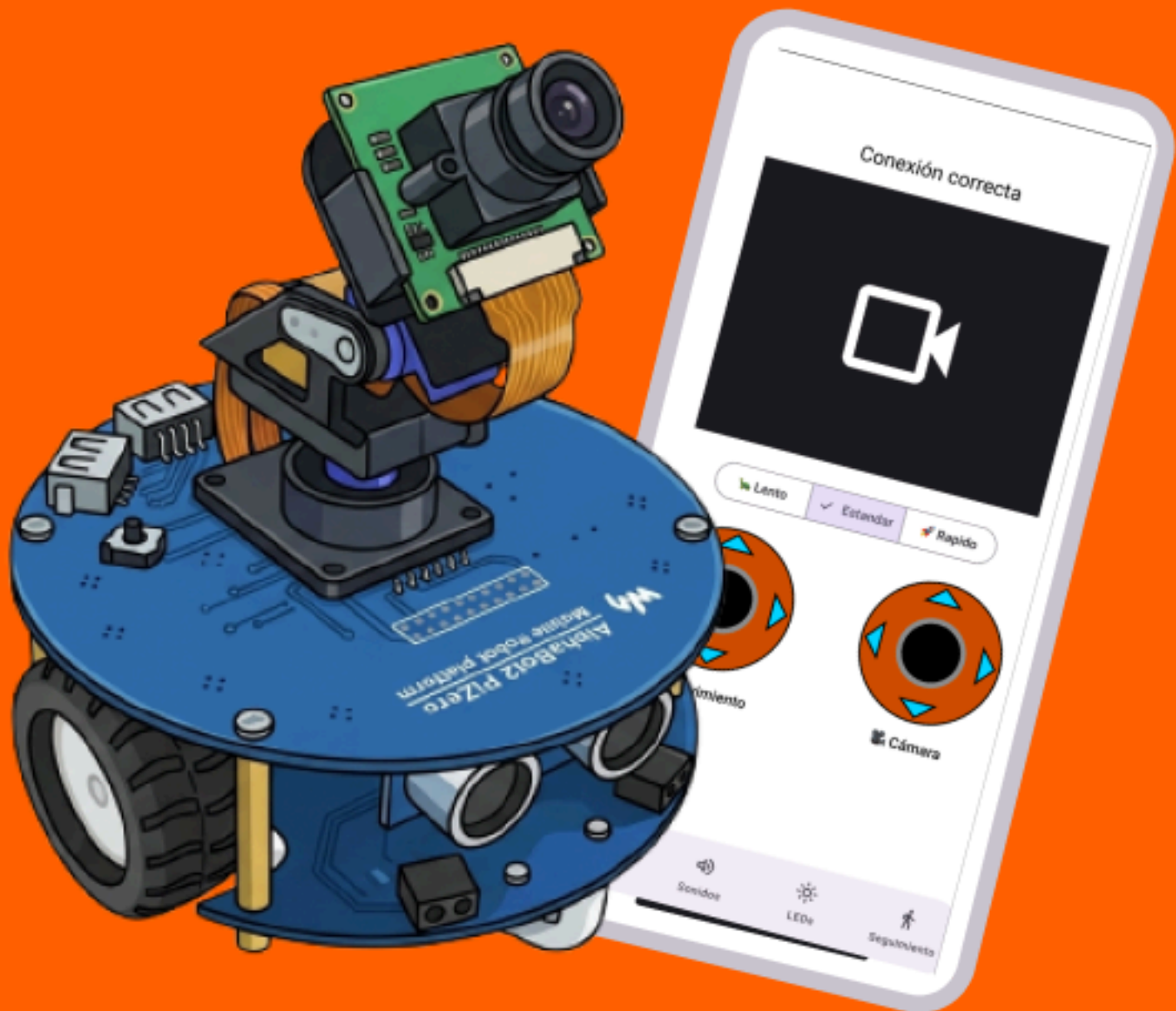


APP ALPHABOT2 TFG – C.F.G.S DAM



JOSE MANUEL LUQUE GONZALEZ
I.E.S. RAFAEL ALBERTI



1. Introducción.....	3
1.1. Descripción General.....	3
1.2. Objetivos.....	3
1.3. Tecnologías utilizadas.....	3
1.4. Antecedentes.....	4
2. Descripción.....	4
2.1. Funcionalidades.....	4
3. Instalación y Preparación.....	4
4. Prototipado.....	4
5. Diseño Funcional.....	5
5.1 Descripción funcional.....	5
5.2 Diagrama de casos de uso.....	6
5.3 Diagramas de actividad.....	7
5.3.1 Conexión SSH.....	7
5.3.2 Proceso de autenticación.....	8
5.4 Diagrama de secuencia.....	9
5.4.1 Movimiento del robot.....	9
5.5 Diagrama Entidad Relación.....	10
6. Desarrollo.....	10
6.1. Desarrollo por fases.....	10
6.1.1. Scripts del robot (Raspberry Pi).....	10
6.1.2. Conexión y comunicación con el robot (SSH + servicios).....	11
6.1.3. Control del robot (joysticks táctiles + mando Bluetooth).....	11
6.1.4. Sensores y funcionalidades del robot (GPIO / Waveshare).....	11
6.1.5. Cámara (funcional, pero con problemas físicos y de rendimiento).....	11
6.1.6. Resto de control: sonidos, LEDs y seguimiento de línea.....	11
6.1.7. Firebase (usuarios y autenticación) + parte social (posts).....	12
6.2. Dificultades encontradas y decisiones afrontadas.....	12
6.3. Control de versiones y revisión de código.....	12
7. Pruebas.....	12
7.1. Pruebas de Integración (App - Firebase).....	12
7.2. Pruebas de Hardware y Conectividad (App - AlphaBot2).....	13
7.3. Pruebas de Interfaz y Usabilidad (UX/UI).....	13
8. Distribución.....	13
8.1. Tecnología de distribución.....	13
8.2. Proceso de compilación y firmado.....	14
8.3. Canales de distribución.....	14
9. Manual.....	14
10. Conclusiones y Mejoras.....	15
11. Bibliografía y referencias.....	15

1. Introducción

1.1. Descripción General

App AlphaBot2 es una aplicación móvil desarrollada para Android cuyo objetivo principal es facilitar el control integral del robot educativo AlphaBot2 de WaveShare desde una única interfaz. La aplicación nace de la necesidad de evitar el uso de scripts sueltos ejecutados desde un ordenador, centralizando en una sola app tanto el control del robot como la interacción entre usuarios. Además, incorpora una comunidad en la que los usuarios pueden publicar dudas, comentarios y soluciones para ayudarse entre sí.

1.2. Objetivos

El proyecto persigue varios objetivos principales:

- Permitir el control remoto del robot AlphaBot2 desde un dispositivo Android.
- Unificar en una sola aplicación todas las funciones necesarias para manejar el robot.
- Ofrecer una comunidad de usuarios para compartir conocimientos, resolver problemas y publicar contenido relacionado con el robot.
- Mejorar la experiencia de uso frente al control tradicional basado en scripts aislados.
- Desarrollar una aplicación moderna utilizando herramientas actuales de desarrollo móvil.

1.3. Tecnologías utilizadas

Para la realización del proyecto se han utilizado las siguientes tecnologías:

- Python: Para los scripts encargados de controlar el robot.
- Kotlin: Como lenguaje principal de la aplicación móvil.
- Android Studio: Como entorno de desarrollo.
- Jetpack Compose: Para construir la interfaz de usuario de forma declarativa.
- Github: Para el control de versiones tanto de la app como de los scripts del robot.
- Firebase: Tanto para la autenticación como la base de datos documental y una base de datos en la nube para las imágenes de usuarios y posts..

- Figma: Para el prototipado de la aplicación

1.4. Antecedentes

Durante la elaboración del proyecto se identificó la existencia de una aplicación oficial para el robot AlphaBot2, aunque esta no resultaba funcional en el momento de desarrollar la idea, ya que sus servidores estaban caídos y la aplicación era inutilizable. A partir de esa situación, y tras la experiencia adquirida durante la FCT en el departamento de robótica de la ESI, surgió la idea de crear una solución propia que simplificara el control del robot.

La propuesta nace al observar cómo estaban programados los robots en la página oficial y al comprobar que era posible desarrollar scripts personalizados para manejar sus funciones de una forma más cómoda y accesible. Por ello, App AlphaBot2 se plantea como una alternativa más práctica, que permite controlar el robot desde una única aplicación y, además, incorpora una comunidad de usuarios para compartir ayuda e información.

2. Descripción

2.1. Funcionalidades

- Registrarse e iniciar sesión.
- Conectarte al robot mediante wifi.
- Controlar todos los aspectos y controles disponible que tiene el robot
- Control con un mando (Testeado con un mando de PS5) del robot conectando el mando por bluetooth al dispositivo móvil.
- Visualizar y editar todos los datos del perfil del usuario incluyendo subir una foto para el avatar.
- También podemos publicar posts y borrar los nuestros.
- Comentar y dar me gusta en los posts de otros usuarios.
- Puedes ser Administrado y borrar cualquier publicación además de bloquear temporalmente usuarios o eliminar cuentas

3. Entorno de Ejecución y Requisitos Técnicos

El proceso detallado de instalación, configuración de IPs estáticas y despliegue del software se encuentra documentado en el Anexo correspondiente al [Manual de Instalación y Uso](#) del proyecto. A continuación, se detallan los requisitos técnicos y la arquitectura de hardware y software necesaria para soportar el ecosistema de la aplicación.

3.1. Requisitos de Hardware

- **Robot AlphaBot2:** Equipado con una placa controladora (Raspberry Pi Zero 2W), motores de corriente continua, servos para el movimiento de la cámara , sensores de infrarrojos, un buzzer y 4 luces leds.
- **Dispositivo Cliente:** Smartphone o tablet con sistema operativo Android.
- **Periféricos (Opcional):** Mando de control físico compatible con conectividad Bluetooth para el mapeo de los ejes y botones en la aplicación.

3.2. Infraestructura de Red

- Conexión de red de área local (Wi-Fi) compartida entre el robot y el dispositivo Android para permitir la apertura y comunicación bidireccional mediante Sockets TCP y el protocolo SSH.

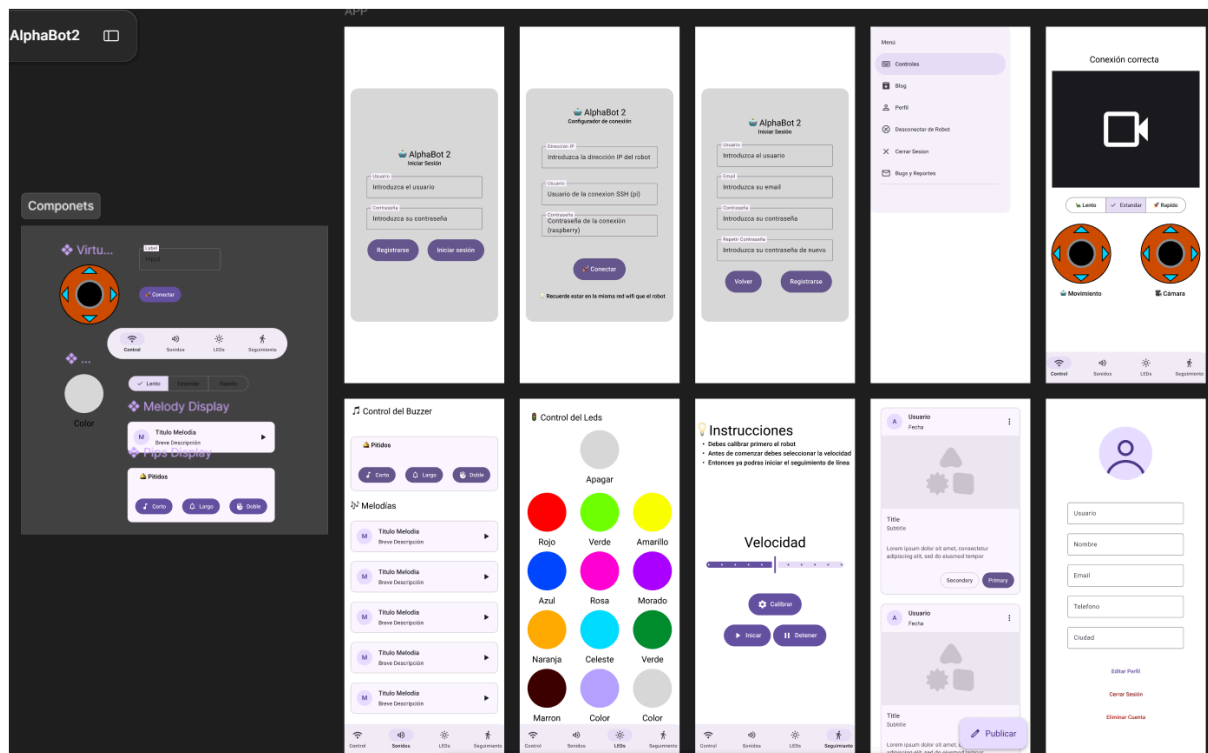
3.3. Requisitos de Software y Servicios Cloud

- **Backend (Firebase):** Proyecto activo en Google Firebase configurado con los servicios de Authentication, Cloud Firestore (para la base de datos documental) y Cloud Storage (para el almacenamiento de imágenes de perfil y publicaciones).
- **Entorno Robot (Raspberry Pi OS):** Intérprete de Python 3 instalado junto con las librerías `RPi.GPIO` y `rpi_ws281x` para la manipulación directa de los pines de la placa.

4. Prototipado

El prototipo inicial de la aplicación está realizado en la plataforma Figma donde se pueden ver todas las pantallas disponibles a parte de los componentes creados para la misma, adjunto un enlace público y una captura.

[Enlace a Figma](#)



5. Diseño Funcional

5.1 Descripción funcional

La aplicación AlphaBot2 permite controlar remotamente un robot AlphaBot2 basado en Raspberry Pi desde un dispositivo Android.

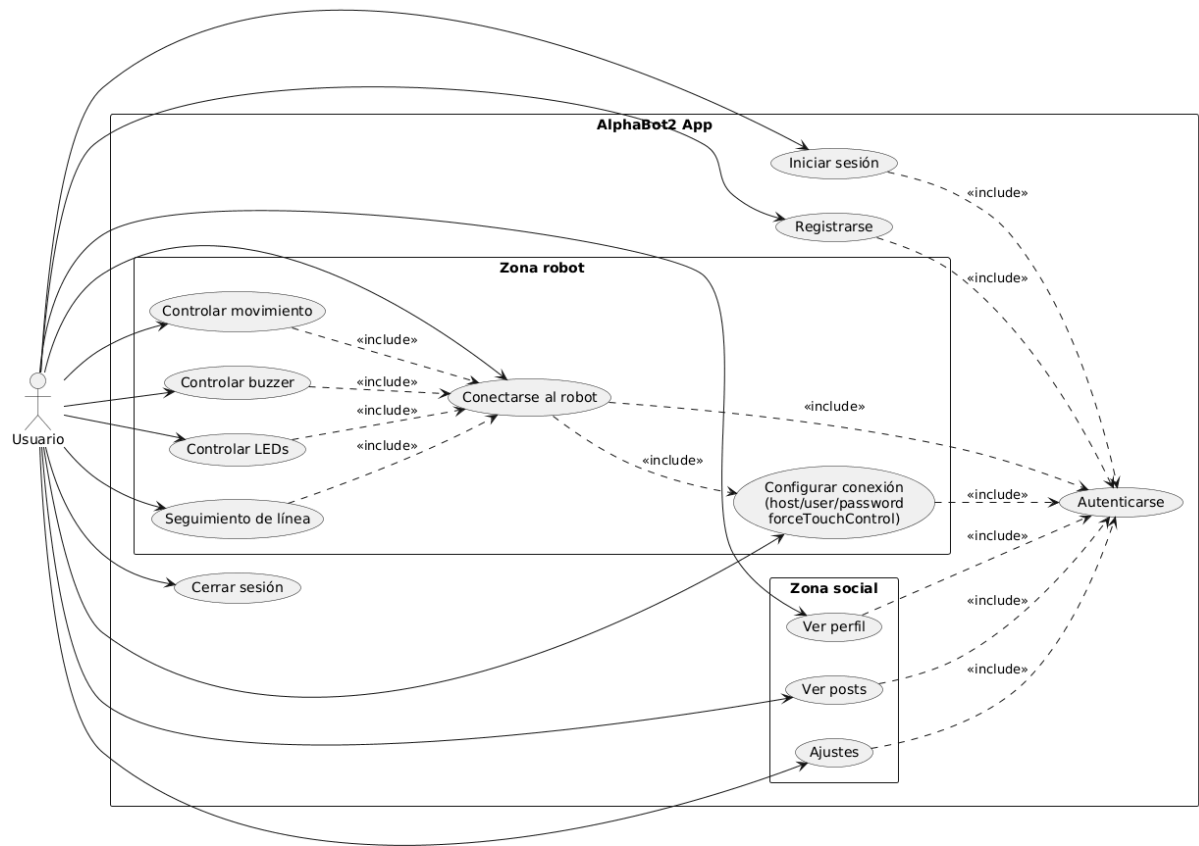
El usuario puede autenticarse en la aplicación, configurar los datos de conexión del robot y establecer comunicación con la Raspberry Pi mediante SSH. Una vez conectada, la aplicación inicia automáticamente los servicios necesarios para el funcionamiento del robot.

A través de la interfaz principal el usuario puede:

- Controlar el movimiento del robot.
- Visualizar la transmisión de vídeo en tiempo real.
- Activar sonidos mediante el buzzer.
- Controlar los LEDs.
- Activar el modo de seguimiento de línea.(No funciona correctamente)
- Consultar y gestionar información de usuario.
- Compartir publicaciones y comentarios con otros usuarios.

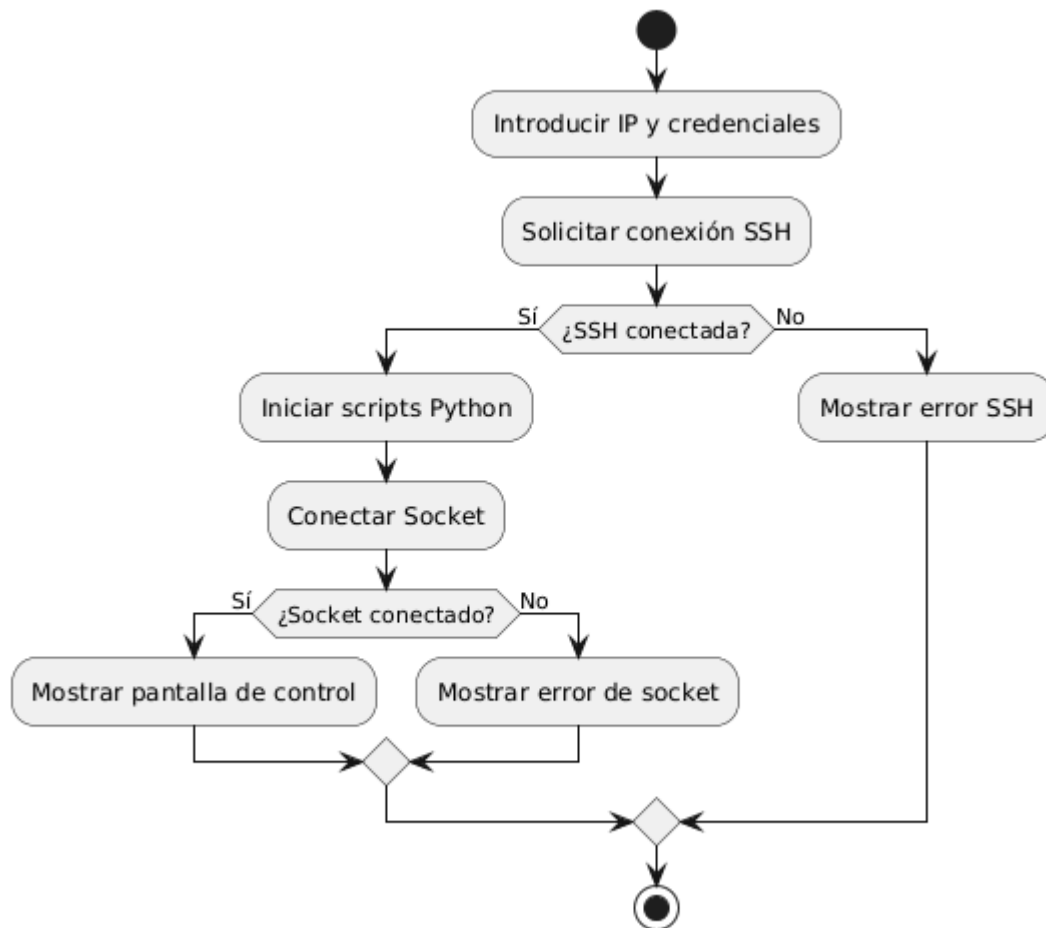
La arquitectura se basa en una aplicación Android desarrollada con Kotlin y Jetpack Compose que se comunica con diversos scripts Python ejecutados en la Raspberry Pi del robot.

5.2 Diagrama de casos de uso

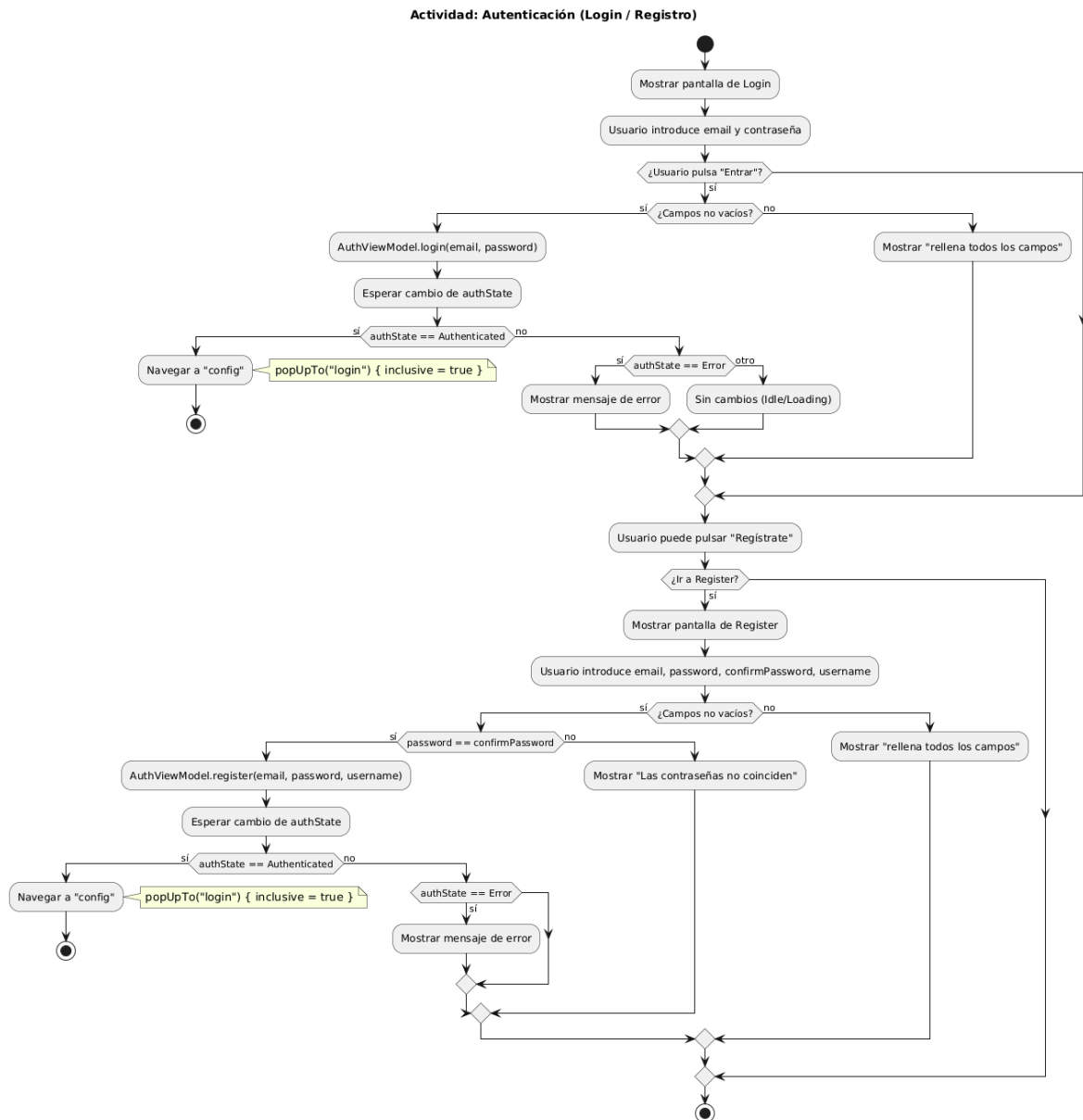


5.3 Diagramas de actividad

5.3.1 Conexión SSH

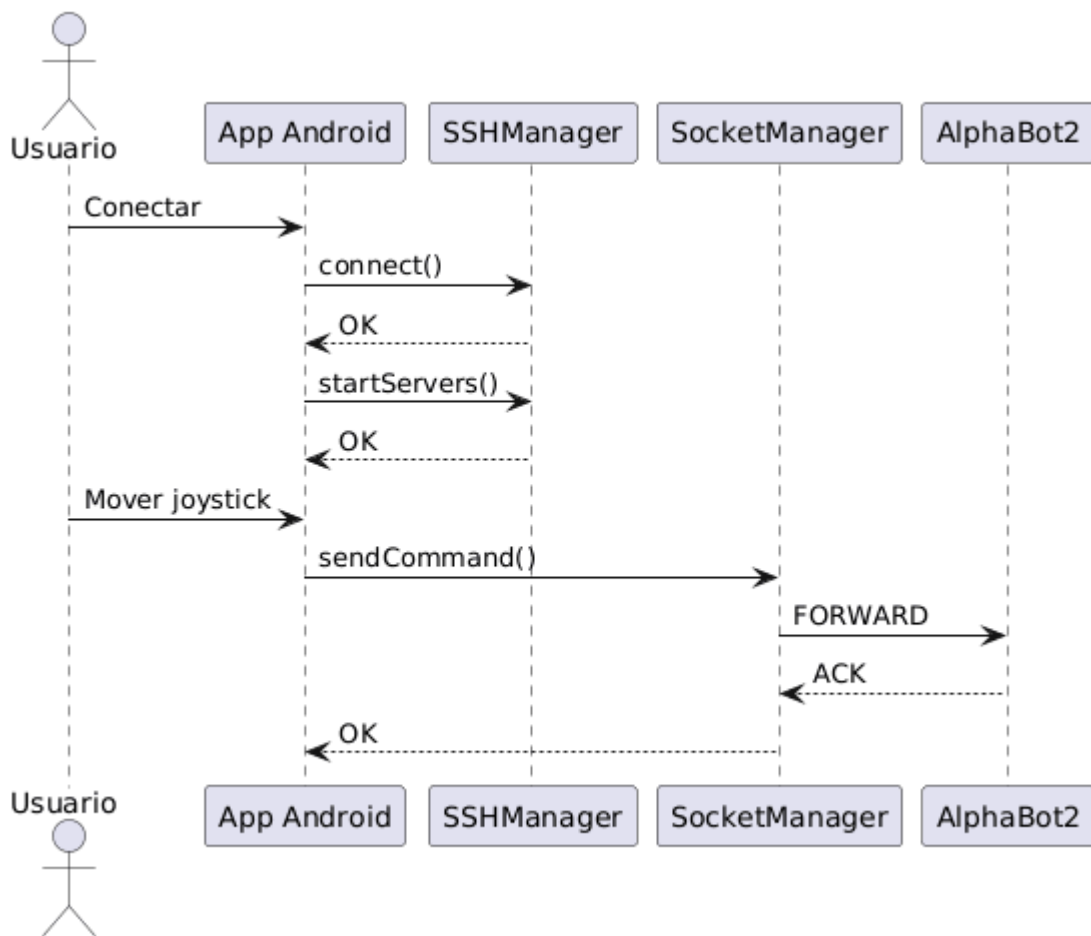


5.3.2 Proceso de autenticación

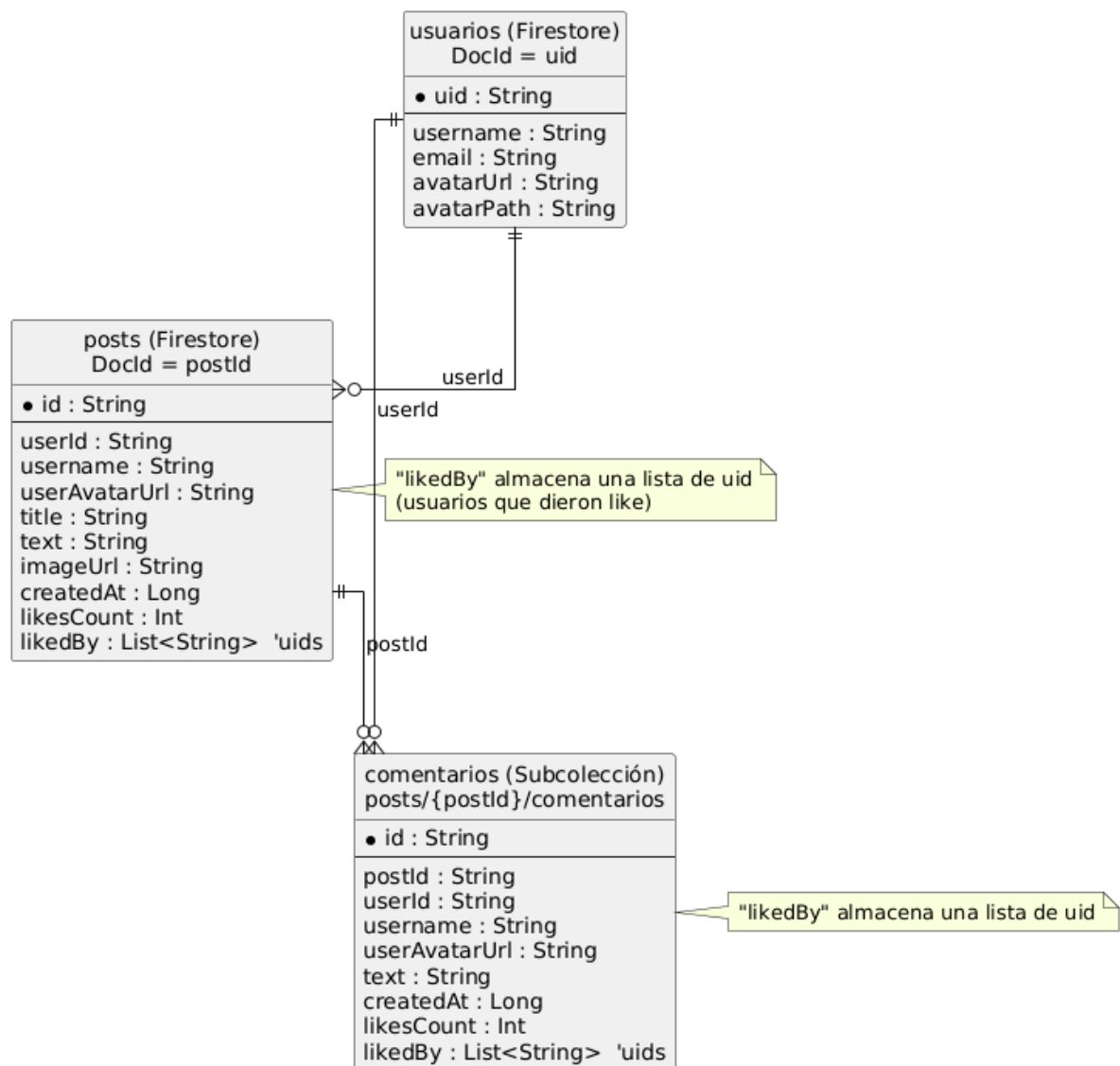


5.4 Diagrama de secuencia

5.4.1 Movimiento del robot



5.5 Diagrama Entidad Relación



Las imágenes estarán disponibles en el repositorio en una carpeta llamada Diagramas por si no se aprecian del todo bien en este documento-

[Enlace a los Diagramas en github](#)

6. Desarrollo

El desarrollo lo hice por fases, empezando por lo más básico para poder probar cuanto antes y luego añadiendo funcionalidades poco a poco, ha sido un desarrollo bastante largo dado mi desconocimiento sobre la materia.

6.1. Desarrollo por fases

6.1.1. Scripts del robot (Raspberry Pi)

Antes de centrarme en la aplicación Android, preparé los scripts que se ejecutan en el robot en un repositorio aparte ([Repo de Scripts](#)) para poder actualizarlos cuando sea necesario directamente con un pull. La idea era tener una base estable en la Raspberry para poder ir comprobando cada funcionalidad desde la app.

6.1.2. Conexión y comunicación con el robot (SSH + servicios)

El primer reto real en la app fue conseguir una conexión SSH estable usando una librería y poder ejecutar/gestionar comandos en la Raspberry. Esta parte fue clave porque sin ella no podía lanzar servidores ni depurar bien el resto.

6.1.3. Control del robot (joysticks táctiles + mando Bluetooth)

Después implementé el control del robot. Para los joysticks táctiles utilicé una librería/proyecto ya existente, y mi trabajo consistió en integrar esa librería y traducir sus valores de entrada a comandos entendibles por el robot (los créditos los incluyo al final del documento).

Además, añadí soporte para mando Bluetooth, para poder controlar el robot con el gamepad (Únicamente probado con un mando de PS5) cuando esté disponible.

Durante esta fase me encontré un problema concreto: en algunos móviles (por ejemplo POCO) el sistema detectaba “algo” como mando aunque no hubiera nada conectado, y por eso no aparecían bien los controles táctiles. Para solucionarlo añadí en la pantalla de configuración la opción “Forzar control táctil”, que obliga a ignorar el mando y usar siempre la pantalla.

6.1.4. Sensores y funcionalidades del robot (GPIO / Waveshare)

Para entender bien cómo trabajar con los sensores a nivel de programación (GPIO), me apoyé en la documentación oficial de Waveshare. Esto me ayudó a tomar decisiones sobre cómo lanzar/organizar los scripts y cómo interpretar los datos del robot.

6.1.5. Cámara (funcional, pero con problemas físicos y de rendimiento)

La cámara fue una de las partes más duras: tanto por el montaje del servidor/streaming como por las limitaciones reales del hardware. En mi caso hay dos causas principales de problemas:

- Potencia limitada de la Raspberry Pi Zero 2W (se nota en rendimiento/latencia).
- Cable flex de la cámara, que no siempre hace buen contacto: el código funciona, pero a veces la cámara falla por motivos físicos (por ejemplo al mover el robot o el cable).

6.1.6. Resto de control: sonidos, LEDs y seguimiento de línea

Fui añadiendo el resto de módulos de control (sonidos y LEDs) como funcionalidades independientes dentro de la app.

El seguimiento de línea está integrado, pero actualmente no funciona correctamente, así que lo dejo documentado como un problema pendiente de depurar/mejorar.

6.1.7. Firebase (usuarios y autenticación) + parte social (posts)

Al principio la app no tenía persistencia, así que más adelante integré Firebase para gestionar usuarios y autenticación. Una vez esa base estaba lista, implementé la parte social (posts e interacción) sobre Firestore/Storage.

6.2. Dificultades encontradas y decisiones afrontadas

- **La comunicación con el robot** (especialmente la conexión SSH y el control en tiempo real) fue lo que más esfuerzo requirió al inicio, porque condiciona todo lo demás.
- **La cámara** fue especialmente problemática por el servidor y por las limitaciones del hardware (Raspberry Pi Zero 2W) y del propio cableado (flex).
- **Integración de joysticks:** en vez de desarrollar unos joysticks desde cero, decidí usar una librería existente y centrarme en lo importante: integración, mapeo de controles y funcionamiento real con el robot (créditos incluidos en el documento).
- **Compatibilidad entre dispositivos:** el problema detectado en móviles POCO con el mando Bluetooth me obligó a tomar una decisión práctica: añadir un “modo forzar táctil” para asegurar que el usuario siempre pueda controlar el robot aunque el sistema detecte un dispositivo de entrada incorrectamente.
- **Alcance/tiempo:** el seguimiento de línea queda como funcionalidad implementada a nivel de interfaz e integración, pero con fallos en la práctica; preferí dejar el resto estable y documentar esta parte como mejora pendiente.

6.3. Control de versiones y revisión de código

He utilizado Git y GitHub para controlar versiones, haciendo commits frecuentes para mantener un histórico claro y poder volver atrás si algún cambio rompía algo.

También separé el trabajo en dos repositorios (scripts del robot y app Android) para mantener el proyecto más ordenado. Para revisar el código me apoyé en el historial de commits y en las diferencias (diff) de GitHub antes de seguir avanzando con nuevas funcionalidades.

7. Pruebas

En este apartado se describen los protocolos seguidos para garantizar la estabilidad de la aplicación y la correcta comunicación con el hardware y los servicios en la nube.

7.1. Pruebas de Integración (App - Firebase)

Se realizaron pruebas para verificar la persistencia de datos y la seguridad en la autenticación:

Autenticación: Validación de flujos de registro e inicio de sesión mediante Firebase Auth. Se comprobó el manejo de errores ante credenciales incorrectas.

Sincronización en tiempo real: Verificación de la latencia en la publicación de posts y comentarios en Cloud Firestore, asegurando que los datos se actualicen correctamente en la interfaz de todos los usuarios.

7.2. Pruebas de Hardware y Conectividad (App - AlphaBot2)

Dada la naturaleza del proyecto, este bloque fue crítico:

Control de Latencia: Pruebas de envío de comandos de movimiento para asegurar que el tiempo de respuesta entre la pulsación en la app y el movimiento del robot sea mínimo.

Pruebas de Regresión: Tras cada modificación en los scripts de Python del robot, se verificó que las funciones básicas (avanzar, girar, detenerse) siguieran operativas desde la interfaz de Android.

7.3. Pruebas de Interfaz y Usabilidad (UX/UI)

Se ejecutaron pruebas en distintos dispositivos físicos y emuladores para asegurar:

Adaptabilidad: Correcto renderizado de los componentes de Jetpack Compose en diferentes resoluciones de pantalla.

Navegación: Verificación de que el NavController gestiona correctamente la pila de pantallas, evitando cierres inesperados al usar el botón de retroceso.

8. Distribución

En este apartado se describe el proceso técnico para transformar el código fuente en un producto instalable y los canales utilizados para su entrega.

8.1. Tecnología de distribución

Para la generación del paquete instalable se ha utilizado el formato APK (Android Package Kit). Aunque Google recomienda el uso de Android App Bundles (.aab) para la Play Store, se ha optado por el APK debido a su facilidad para la instalación directa y su compatibilidad con repositorios alternativos.

8.2. Proceso de compilación y firmado

Generación del APK firmado: Se utiliza el asistente de generación de paquetes de Android Studio, empleando una clave de firmado digital (Keystore) generada específicamente para este proyecto. Esto garantiza la integridad de la app y que no pueda ser suplantada por una versión maliciosa.

8.3. Canales de distribución

Dado que el proyecto tiene un enfoque educativo y se basa en hardware específico (AlphaBot2), se han definido dos vías de distribución:

Distribución Directa (GitHub): El APK se aloja en la sección de Releases del repositorio oficial del proyecto, permitiendo a los desarrolladores acceder tanto al código como al binario.

[Enlace a las Releases](#)

Repositorio Alternativo (APKPure): Se ha seleccionado APKPure como plataforma de distribución externa. Esta decisión se basa en:

- Accesibilidad: Permite la descarga sin necesidad de una cuenta de Google activa.
- Visibilidad: Facilita que otros entusiastas de la robótica encuentren la herramienta fuera del entorno de desarrollo.
- Control de versiones: La plataforma permite gestionar distintas versiones de la app, facilitando que el usuario final reciba actualizaciones.
- Precio: Es una alternativa que me permitía subir el APK de forma completamente gratuita.

Para ello he necesitado hacer los terminos y condiciones además de la política de privacidad para que me aprobaran la app, una vez tuve todos los requisitos me la aprobaron y aquí está el enlace:

[Enlace APKpure](#)

9. Manual

El manual de instalación y uso está creado en otro documento aparte para poder subirlo en el proyecto de forma independiente.

[Enlace al otro documento](#)

10. Conclusiones y Mejoras

10.1. Conclusiones

El desarrollo de la aplicación cumple con el objetivo principal de ofrecer una interfaz centralizada y nativa para el control del AlphaBot2. La utilización de Jetpack Compose ha permitido una construcción modular de la interfaz de usuario. La implementación de comunicaciones híbridas (conexiones SSH mediante JSch para comandos de sistema y Sockets TCP para el control de latencia en el movimiento y los LEDs) resulta efectiva para el manejo en tiempo real. Asimismo, la integración de Firebase dota a la plataforma de una capa social y de gestión de usuarios funcional y segura.

10.2. Mejoras Futuras

- **Internacionalización Completa:** Conectar el cambio de estado visual del idioma en la pantalla de configuración con el sistema de recursos de Android (strings.xml) para ofrecer soporte multi idioma real.
- **Optimización de Streaming de Vídeo:** Implementar protocolos de transmisión de baja latencia (como WebRTC) para sustituir o mejorar el flujo actual de la cámara.
- **Despliegue en el Ecosistema iOS:** Actualmente, el proyecto es exclusivo de la plataforma Android. Una mejora prioritaria es la migración o adaptación del código para lanzar una versión nativa para dispositivos Apple (iOS), evaluando tecnologías como Kotlin Multiplatform para aprovechar la lógica de negocio ya desarrollada.
 -
- **Panel de Administración del Sistema (Raspberry Pi):** Desarrollar un módulo de administración avanzado dentro de la aplicación con privilegios elevados. Este panel permitiría actualizar los scripts directamente desde github, monitorizar el estado de la Raspberry Pi (temperatura del procesador, uso de memoria RAM), gestionar el ciclo de vida de los scripts de Python (iniciar, detener o reiniciar servicios) y realizar reinicios del sistema operativo de forma remota y gráfica, eliminando la necesidad de usar una consola externa.
- **Depuración y Activación del Seguimiento de Línea:** El código de comunicación por red (puerto 5003) para el módulo sigue-líneas se encuentra implementado en la

aplicación. La mejora consistirá en depurar la algoritmia de lectura de los sensores infrarrojos físicos y el control PWM de los motores para que el robot sea capaz de trazar el recorrido de forma fluida y autónoma.

11. Bibliografía y referencias

- **Google Developers:** Documentación oficial de Android Developers y Jetpack Compose.
- **Firebase Documentation:** Guías de integración y uso para Firebase Authentication, Cloud Firestore y Cloud Storage en plataformas Android.
- **Waveshare:** Wiki y manuales técnicos oficiales del hardware AlphaBot2 para Raspberry Pi.
- **JSch:** Documentación oficial de la librería JCraft (JSch) para la implementación de sesiones y ejecución de comandos SSH en Java/Kotlin.
- **JetStick:** Librería usada para la implementación de los joysticks virtuales
- **UCrop:** Librería usada para recortar los avatares de usuario